

DataScript — Design Specification (D1)

CSE 341 · Part 1 · Due 8 May 2026

4.1 Language Overview [P1]

Language name: DataScript

Intended domain: Querying and transforming structured tabular data (CSV files or in-memory tables).

Sample program:

```
table students = load "students.csv"

let result = SELECT name, grade
              FROM students
              WHERE grade > 70 AND grade < 90
              ORDER BY grade DESC

print result
```

Justification (Sebesta Ch. 1 language evaluation criteria):

DataScript prioritizes **readability** (§1.3.1) and **writability** (§1.3.2) for its target domain. Readability is achieved through SQL-inspired syntax: a program reads almost like a natural-language query, so a domain user — a data analyst who is not a programming-language expert — can understand what a program does without prior training. Writability is served by domain-specific constructs (SELECT, WHERE, GROUP BY) that express common data operations in a single declarative statement rather than requiring explicit loops and temporary variables.

DataScript partially prioritizes **reliability** (§1.3.3) through static strong typing: column types are determined when a table is loaded, and a type mismatch (e.g., comparing a string column with an integer literal) is caught before execution rather than producing a silent wrong result.

Cost (§1.3.4) is knowingly sacrificed. DataScript is not general-purpose: it cannot perform file I/O beyond table loading, has no network access, and offers no general recursion or imperative side effects. This is an intentional design trade-off — following Sebesta's observation that special-purpose languages (§1.2) gain expressiveness in their domain precisely by surrendering generality outside it.

4.2 Lexical Structure [P1]

Token categories are defined using regular expressions. Keywords are case-sensitive and always uppercase (e.g., SELECT, FROM, WHERE). Identifiers are case-sensitive and always start with a lowercase letter or

underscore. This asymmetry is a deliberate reliability decision: it makes keywords visually distinct from user-defined names and prevents silent errors from mixed-case typos.

Token Categories

Category	Pattern (regex)	Examples
IDENT	<code>[a-zA-Z_][a-zA-Z0-9_]*</code>	<code>students</code> , <code>grade</code> , <code>my_table</code>
INT_LIT	<code>[0-9]+</code>	<code>0</code> , <code>42</code> , <code>100</code>
FLOAT_LIT	<code>[0-9]+\.[0-9]+</code>	<code>3.14</code> , <code>0.5</code> , <code>99.0</code>
STRING_LIT	<code>"[^\\n]*"</code>	<code>"hello"</code> , <code>"students.csv"</code>
BOOL_LIT	<code>true</code> <code>false</code>	<code>true</code> , <code>false</code>

Keywords

Keywords are reserved identifiers — they match the IDENT pattern but are pre-empted by the lexer before identifier matching. This is standard practice (Sebesta §3.1).

SELECT	FROM	WHERE	ORDER	BY	GROUP
HAVING	AND	OR	NOT	ASC	DESC
COUNT	SUM	AVG	MIN	MAX	
table	load	let	print		
if	then	else	function	return	
int	float	string	bool		
true	false				

`true` and `false` are reserved as keywords so that they cannot be used as identifier names. The lexer emits a BOOL_LIT token (not IDENT) when it encounters them, using the same keyword pre-emption logic applied to all other reserved words.

Operators

Token	Lexeme	Notes
EQ	<code>==</code>	equality comparison
NEQ	<code>!=</code>	inequality comparison
LTE	<code><=</code>	less than or equal
GTE	<code>>=</code>	greater than or equal
LT	<code><</code>	less than
GT	<code>></code>	greater than
ASSIGN	<code>=</code>	variable binding (let x = ...)
PLUS	<code>+</code>	addition

Token	Lexeme	Notes
MINUS	-	subtraction / unary negation
STAR	*	multiplication / SELECT all cols
SLASH	/	division
ARROW	->	function return type annotation

Note: <=, >=, !=, == and -> are scanned as single tokens before their single-character prefixes (<, >, !=, =, -). The lexer applies maximal munch (longest match first).

Separators

Token	Lexeme	Usage
LPAREN	(	function call, grouping
RPAREN	)	function call, grouping
LBRACE	{	function body
RBRACE	}	function body
COMMA	,	column list, parameter list
COLON	:	type annotation (param: type)

Whitespace and Comments

Whitespace ([\t\n\r]+) is ignored between tokens. Single-line comments begin with -- and extend to end of line: --[^n]*. There are no multi-line comments.

4.3 Syntax [P1]

The complete grammar is given in EBNF (Sebesta §3.1–3.2). Terminals are written in double quotes or as token category names in ALL_CAPS. { X } means zero or more repetitions of X; [X] means X is optional; (X | Y) means alternation.

Operator precedence is encoded by grammar layering: lower-precedence operators appear in higher-level non-terminals. This is the standard technique described in Sebesta §3.4 — a grammar that encodes precedence is unambiguous by construction.

Program and Statements

```
<program>      ::= { <statement> }

<statement>    ::= <table_decl>
                  | <let_stmt>
                  | <print_stmt>
                  | <if_stmt>
```

```

    | <func_decl>
    | <return_stmt>

<table_decl> ::= "table" IDENT "=" "load" STRING_LITERAL

<let_stmt>   ::= "let" IDENT "=" <expr>

<print_stmt> ::= "print" <expr>

<if_stmt>    ::= "if" <expr> "then" <block> [ "else" <block> ]

<func_decl>  ::= "function" IDENT "(" [ <param_list> ] ")" "->" <type> <block>

<param_list> ::= <param> { "," <param> }
<param>      ::= IDENT ":" <type>

<return_stmt> ::= "return" <expr>

<block>       ::= "{" { <statement> } "}"

<type>        ::= "int" | "float" | "string" | "bool" | "table"

```

Dangling-else resolution: DataScript requires `{ }` braces around every branch of an `if` statement. Because branch bodies are always explicitly delimited blocks, the dangling-else problem does not arise at all — the grammar is unambiguous by construction. In languages where branches can be bare statements (e.g., C), "else binds to nearest if" is a conventional disambiguation rule. DataScript avoids needing that rule entirely.

Expression Grammar (Operator Precedence)

Precedence is encoded by grammar layers. Each level can only refer downward; the lowest non-terminal in the chain has the highest binding power.

```

<expr>      ::= <or_expr>

<or_expr>   ::= <and_expr> { "OR" <and_expr> }

<and_expr>  ::= <not_expr> { "AND" <not_expr> }

<not_expr>  ::= "NOT" <not_expr>
              | <compare_expr>

<compare_expr> ::= <add_expr> [ ( "==" | "!=" | "<" | "<=" | ">" | ">=" )
<add_expr> ]

<add_expr>  ::= <mul_expr> { ( "+" | "-" ) <mul_expr> }

<mul_expr>  ::= <unary_expr> { ( "*" | "/" ) <unary_expr> }

<unary_expr> ::= "-" <unary_expr>
              | <primary>

```

```
<primary> ::= INT_LIT
           | FLOAT_LIT
           | STRING_LIT
           | BOOL_LIT
           | <aggregate>
           | <func_call>
           | <query_expr>
           | IDENT
           | "(" <expr> ")"

<func_call> ::= IDENT "(" [ <arg_list> ] ")"
<arg_list>  ::= <expr> { "," <expr> }

<aggregate> ::= ( "COUNT" | "SUM" | "AVG" | "MIN" | "MAX" ) "(" ( IDENT | "*" )
                ")"
```

Associativity: { } in EBNF iterates left to right, producing left-associative parsing for +, -, *, /, OR, and AND. For example, a - b - c parses as (a - b) - c. Comparison operators (==, !=, <, etc.) are non-associative — chaining a < b < c is a syntax error by design, preventing the common mistake of writing range checks incorrectly.

Unary minus is right-associative via recursion: --x parses as -(-(x)).

Query Expression (Domain-Specific Construct)

The query expression is the defining construct of DataScript. It is an expression (not a statement) because it produces a value of type `table`, which can be bound with `let` or passed to a function.

```
<query_expr> ::= "SELECT" <select_list>
               "FROM" IDENT
               [ "WHERE" <expr> ]
               [ "GROUP" "BY" <col_list> [ "HAVING" <expr> ] ]
               [ "ORDER" "BY" <order_list> ]

<select_list> ::= "*" | <col_expr> { "," <col_expr> }
<col_expr>    ::= IDENT | <aggregate>
<col_list>    ::= IDENT { "," IDENT }
<order_list>  ::= <order_item> { "," <order_item> }
<order_item>  ::= IDENT [ "ASC" | "DESC" ]
```

Operator Precedence Summary (highest to lowest)

Level	Operator(s)	Associativity
1	unary -	right
2	* /	left
3	+ -	left

Level	Operator(s)	Associativity
4	<code>==</code> <code>!=</code> <code><</code> <code><=</code> <code>></code> <code>>=</code>	non-associative
5	<code>NOT</code>	right (prefix)
6	<code>AND</code>	left
7	<code>OR</code>	left

4.6 Names, Binding, Scope, Lifetime [P1]

Legal Identifiers

An identifier must match `[a-zA-Z_][a-zA-Z0-9_]*` and must not be a reserved keyword (see §4.2). Identifiers are case-sensitive: `grade`, `Grade`, and `GRADE` are three distinct names. The underscore prefix (e.g., `_tmp`) is allowed but conventionally reserved for implementation-internal names.

Bindings: Compile Time vs Run Time (Sebesta §5.4)

A binding is an association between a name and an attribute such as type, value, or storage location. Binding time determines when that association is fixed.

Compile-time bindings in DataScript:

- The types of function parameters and return values, declared explicitly with `:` and `->`. Example: `function avg_grade(t: table) -> float` — the types of `t` and the return value are bound before execution.
- The meaning of keywords and operators (e.g., `AND` always means logical conjunction).

Runtime bindings in DataScript:

- The value of any `let`-bound variable — e.g., `let x = 5` binds `x` to `5` when that line executes.
- The schema (column names and types) of a loaded table. When `table students = load "students.csv"` executes, the interpreter reads the CSV header and infers column types at that moment. This is a deliberate design decision: requiring the user to declare schemas would improve static type-checking but reduce writability for exploratory use.

Scoping: Static (Lexical) Scoping (Sebesta §5.5)

DataScript uses **static (lexical) scoping**. The scope of a name is determined by its position in the source text, not by the dynamic call stack.

Scoping rules:

- The **global scope** contains all top-level `table` declarations, `function` declarations, and `let` statements.
- Each **function body** introduces a new local scope. Parameters are bound in this scope.
- Each **block** (`{ }`) introduces a nested scope. A `let` inside a block is visible only within that block and any nested blocks.

- Name lookup proceeds inward-to-outward: a name is first looked up in the innermost enclosing scope; if not found, the search continues outward to the global scope.

Why static, not dynamic scoping? Static scoping supports readability (Sebesta §1.3.1): a reader can determine what any name refers to by inspecting the source text alone, without reasoning about the call stack at runtime. Dynamic scoping would mean that a function's behavior could change depending on what variables happen to be in scope at the call site — a serious readability and reliability hazard.

What would break if DataScript used dynamic scoping? Consider:

```
let threshold = 70

function passing(t: table) -> table {
  return SELECT name FROM t WHERE grade > threshold
}

-- later, in a different context:
let threshold = 50
let result = passing(students)  -- with dynamic scoping, uses threshold = 50 !
```

With dynamic scoping, `passing` silently uses the caller's `threshold` instead of the one visible in its definition. With static scoping, `passing` always uses `threshold = 70` regardless of the call site.

Lifetime (Sebesta §5.4.3)

Variable kind	Lifetime	Reason
Global <code>table</code> declaration	Static	Loaded once; available for the whole program.
Global <code>let</code> binding	Static	Bound at top level; lives for the whole run.
Function parameter	Stack-dynamic	Created on function entry, destroyed on return.
<code>let</code> inside a function/block	Stack-dynamic	Created when the binding executes, destroyed when the enclosing block exits.

DataScript has no explicit heap allocation (`new`, `malloc`). There are no pointers or references. Tables returned from a query are treated as values — they do not have independently managed heap lifetimes.